sizeof, Overflow, and Why 0.1 + 0.2 Isn't 0.3

Keep this. You will want the epsilon-comparison pattern and the unsigned-wraparound warning for the rest of the semester — both come back, unannounced, the moment you stop thinking about them.

1. Key terms

Term	Definition
Overflow	A value needs more bits than its type has. The extra bits are silently discarded — the value wraps around. No crash, no warning.
Wraparound	What an unsigned value does instead of going negative: it jumps to the top of its range.
Epsilon	A small tolerance used to compare two doubles for "close enough" instead of using == directly.
Implicit conversion	The compiler silently changing a value's type to make an expression valid — sometimes helpful, sometimes a trap (see §6).
Comparison (relational) operator	==, !=, <, >, <=, >= — compares two values and produces a bool.
Logical operator	&&, ‖, ! — combine or invert bool expressions.
Ternary operator	condition ? a : b — a compact if/else that produces one of two values.
static_cast	An explicit, compile-time-checked conversion: static_cast<Type>(value).
C-style cast	The older explicit-conversion syntax, (Type)value — recognize it in existing code; this course writes static_cast.
Literal	A value written directly into source code (42, 3.14, 'A', "hi", true), rather than read from a variable.
Escape sequence	A backslash followed by a code, standing in for a character that can't be typed directly into a literal (\n, \t, \", \\).
Text encoding	An agreed mapping between numbers and symbols (e.g. ASCII) — a char is really just a small integer under one of these.
ASCII	The 128-code text encoding almost every system agrees on for English letters, digits, and punctuation. 'A' is 65.
Short-circuit evaluation	The guarantee that && stops at the first false operand and ‖ stops at the first true one — the other side is never evaluated.

2. Comparison, logical, and ternary operators

Every comparison produces a bool — true or false. std::cout prints a bool as 1 or 0 by default.

```
==   equal to               a == b
!=   not equal to           a != b
<    less than              a < b
>    greater than           a > b
<=   less than or equal     a <= b
>=   greater than or equal  a >= b
```

> **KEY LINE** == compares; = assigns — two different operators. Confusing them is a classic bug that the compiler will not catch for you (week 3 covers it in depth).

&& (AND) and || (OR) combine two bool expressions; ! (NOT) flips one. Both && and || guarantee the order they check in and can skip the right side entirely — short-circuit evaluation, demonstrated below.

```
int age = 20;
bool isAdult = (age >= 18);
bool hasID   = true;
std::cout << (isAdult && hasID);   // 1
```

Does the right side always run? Not necessarily — C++ guarantees the right side of && or || is skipped entirely once the left side already decides the answer:

```
int a = 3, b = 2, c = 0;
int x = ++a || (++b && ++c);   // ++a makes a=4 (true) -- || short-
                                 // circuits: ++b, ++c NEVER RUN
```

> **KEY LINE** && stops at the first false operand; || stops at the first true one. Any side effect on the skipped side (like ++b here) simply does not happen — this is exactly the guard-condition pattern: denominator != 0 && numerator / denominator > threshold.

The ternary operator condition ? a : b picks one of two values in a single expression:

```
bool isMember = true;
double fee = isMember ? 5.0 : 10.0;
std::cout << fee;   // 5
```

Next week's if/else can write this exact same choice as a full statement instead of a single expression — the ternary form here is the compact preview.

Quick update to the precedence table, now that you have comparison, logical, and ternary operators too: arithmetic, then comparison, then &&, then ||, then ?:, then assignment, then the comma operator.

```
std::cout << (3 + 2 > 4 && 1 < 2);   // 1 -- arithmetic, comparison, then &&
```

3. Char, string, and bool literals

'A' (single quotes) is a char literal — exactly one character. "A" (double quotes) is a string literal — text, even one character long. They are different types:

```
char grade = 'A';           // one character
std::string name = "Ada";   // text -- #include <string>
```

Escape sequences — a backslash followed by a code — stand in for characters that can't be typed directly into a literal:

```
std::cout << "Line one\nLine two\n";    // \n -- newline
std::cout << "She said \"hi\"\n";      // \" -- a literal double-quote
std::cout << "C:\\Users\\ada\n";       // \\ -- a literal backslash
std::cout << "Tab:\there\n";             // \t -- a tab
```

The same escapes work inside a char literal too: '\n' is the newline character, one char, not two.

> **A GLANCE FORWARD** Terminals understand \033[as the start of an ANSI escape code — std::cout << "\033[31mRed\033[0m"; prints Red in red, if the terminal supports it. Not graded this semester.

true and false are literals too — the only two values a bool can hold:

```
bool passed = true;
std::cout << passed;   // 1 -- bool prints as 1/0
```

4. How characters are encoded: ASCII

A char is really just a small integer — a text encoding is the agreed mapping between numbers and symbols. The encoding almost every system agrees on for the basics is ASCII: 'A' is 65, 'a' is 97, '0' is 48.

```
std::cout << 'A' + 0;   // 65 -- adding forces promotion to int
std::cout << '0' + 0;   // 48 -- the CHARACTER '0' is not the NUMBER 0
```

> **KEY LINE** ASCII covers 128 codes (English only). Extended 8-bit encodings (sometimes loosely called "ANSI" on Windows) add 128 more; modern text mostly uses Unicode/UTF-8 instead. This "ANSI" is unrelated to the ANSI escape codes above — same word, two different things.

5. The overflow trap that actually breaks programs

```
unsigned int x = 0;
--x;                 // 4294967295, not -1

for (size_t i = n - 1; i >= 0; --i) {
    // NEVER TERMINATES: i wraps from 0 to a huge
    // number instead of going to -1, and stays >= 0
}
```

> **DEBUGGING, PROPERLY** This bug produces no error and no crash — the only way to catch it is to watch i's actual value in the debugger's Watch panel as it wraps. Reading the code and imagining what it does is not enough for this class of bug.

6. Comparing doubles — and implicit conversions

```
#include <cmath>

double a = 0.1 + 0.2, b = 0.3;
const double EPSILON = 1e-9;
if (std::abs(a - b) < EPSILON) { /* close enough */ }
```

Why == fails here: 0.1 and 0.2 cannot be stored exactly in binary (just as 1/3 cannot be stored exactly in decimal), so their sum is the sum of two tiny approximation errors — not necessarily bit-identical to double's approximation of 0.3.

```
std::cout << 5 / 2;      // 2   -- integer division truncates
std::cout << 5 / 2.0;    // 2.5 -- one operand is double
int total = 5 / 2.0;     // total is 2 -- fraction silently dropped
```

Three kinds of mixing at once, and how the SAME variable can print two different ways:

```
int d1 = 5, d2 = 20;
double x1 = 12.5, x2 = -5.0;
char ch1('*'), ch2('A');
std::cout << d1 + d2;          // 25 -- int + int stays int
std::cout << x1 - x2;          // 17.5 -- double + double stays double
std::cout << ch1 << ch2 + 3;   // *68 -- see below
```

> **KEY LINE** cout << ch1 prints * (the char overload: print the CHARACTER). cout << ch2 + 3 promotes ch2 to int first ('A' is ASCII 65), so it prints 68, not a letter. Same mechanism, different output — because the TYPE changed. Also: x1 and x2 are double straight from their literals, no suffix needed.

7. Explicit conversions: static_cast and the C-style cast

An explicit cast asks for a conversion directly, instead of relying on the implicit rules:

```
int a = 7, b = 2;
double result = static_cast<double>(a) / b;   // 3.5
double result2 = (double)a / b;                // same -- C-style cast
```

> **KEY LINE** static_cast<double>(a) converts a BEFORE the divide runs, so the division is already double / int and promotes to double. static_cast is this course's default — it's checked at compile time and states intent clearly; the C-style cast, (double)a, is older syntax worth recognizing but not writing.
>
> **TODAY'S LAB** Micro-exercise 1, right after this: print type sizes, trigger an overflow on purpose, and compare two doubles the wrong way (==) and the right way (epsilon). Closed-book, no AI, no notes — same rules as the written exam. See the separate Lab handout for the three tasks and how you'll be checked.